



COS110 EO1 2017 - eo test

Computer science (University of Pretoria)



Scan to open on Studocu

Question 1 1.....^{2 1/2}.....(5 marks)

- 1.1 Given the following class with some overloaded constructors and an assignment operator: /Gegewe die volgende klas met sommige oorlaaide konstrueerders en 'n toekenningsoperator:

```
class C {
public:
    C(int = 1) {cout << "A" << endl;}
    C(char) {cout << "B" << endl;}
    C(int, char) {cout << "C" << endl;}
    C(const C&) {cout << "D" << endl;}
    C& operator=(const C&) {cout << "#" << endl;}
};
```

What will the output be for each of the following statements? /Wat sal die afvoer wees vir elkeen van die volgende stellings? (1)

a) C arr1[3];

A
A
A

b) C arr2[] = {'b', 2, 3};

B
A
A

c) C arr3[3] = {C(0, 'Z')};

C
A
A

d) What will the output be for the following statements? /Wat sal die afvoer wees vir die volgende stellings? (2)

```
C * a = new C('p');
C b = *a;
C * d = new C(*a);
b = *d;
```

B
A

D

#

Question 2

Consider the following classes: / Oorweeg die volgende klasse: (17 marks)

```
// Weapon.h:
#ifndef WEAPON.H
#define WEAPON.H
class Weapon
{
    public:
        Weapon(string, int, int);
        ~Weapon();
        string getDescription() const;
        int getWeight() const;
        int getDamage() const;
    private:
        string description;
        int weight;
        int maxDamage;
};
#endif

// Armory.h:
#ifndef ARMORY.H
#define ARMORY.H
#include "Weapon.h"
class Armory
{
    public:
        Armory(int = 10); // size = 10, numWeapons = 0
        Armory(const Armory& rhs);
        Armory& operator=(const Armory& rhs);
        Armory& operator+=(const Weapon& rhs);
        ~Armory();
        const Weapon& getWeapon(string description);
        int getNumWeapons() const;
        int getTotalWeight() const;
    private:
        Weapon ** weapons; // dynamic array of dynamic Weapon objects
                          // NOTE: array may be empty!
        int numWeapons;    // total number of weapons
        int size;          // maximum number of weapons (array size)
};
#endif
```

2.1 What is the relationship between the Armory class and the Weapon class called in the object-oriented programming paradigm? / Wat word die verhouding tussen die Armory klas en die Weapon klas genoem in die objek-geïntereerde paradigma?

Aggregation

(1)

2.2 Implement the = operator for the Armory class. / Implementeer die = operator vir die Armory klas.

(8)

```

Armory & Armory::operator=(const Armory & rhs) {
    numWeapons = rhs.numWeapons;
    size = rhs.size;
    if (this == &rhs) return *this;
    delete for (int i = 0; i < size; i++)
        delete weapons[i];
    delete [] weapons;
    numWeapons = rhs.numWeapons;
    size = rhs.size;
    weapons = new Weapon[size];
}

```

5 1/2

//cont. on other page (5)

2.3 The += operator of the Armory class allows to add new weapons to the Armory objects as follows: / Die += operator van die Armory klas laat toe dat nuwe wapens tot die Armory objekte bygevoeg kan word soos volg:

```

Armory armory;
Weapon sword("Excalibur", 10, 50);
Weapon bow("Long Bow", 5, 30);
armory += sword; // add sword to the armory
armory += bow; // add bow to the armory

```

Implement the += operator to be used as above. New **dynamic copies** of the input objects must be made. The += operator must keep track of the total number of weapons added. If there is no space in the Armory, print the error message: "This armory is full". / Implementeer die += operator sodat dit soos hierbo gebruik kan word. Nuwe dinamiese kopieë van die invoerobjekte moet gemaak word. Die += moet tred hou van die totale aantal wapens wat bygevoeg is. As daar geen spasie in die Armory is nie, moet die volgende foutboodskap uitgedruk word: "This armory is full".

```

Armory & Armory::operator+=(Weapon & w) {
    if (numWeapons < size) {
        weapons[numWeapons] = new Weapon(*w);
        numWeapons++;
    } else {
        cout << "This armory is full" << endl;
    }
    return *this;
}

```

5

2.2. cont. for (int i=0; i~~size~~; i++) {
 weapons[i] = new Weapon(*rhs.weapons[i]);
}
return *this;

}
}

- 2.4 Implement the `getTotalWeight()` function. The function must calculate and return the total weight of the weapons in the Armory. / Implementeer die `getTotalWeight` funksie. Die funksie moet die totale gewig van die wapens in die Armory bereken en terugstuur. (3)

`int w=0;`

```
int Armory::getTotalWeight() const {
    for(int i=0; i<numWeapons; i++) {
        w+= weapons[i] → weapons[i].getWeight();
    }
    return w;
}
```

Question 3 (25 marks)
For all parts of this question, refer to the code below. / Vir alle dele van hierdie vraag, verwys na die kode hieronder.

```
#ifndef ADVENTURER_H
#define ADVENTURER_H
#include "Item.h"
using namespace std;

class Adventurer {
private:
    Item* items;
    string name;
    unsigned int health;
public:
    Adventurer();
    Adventurer(const& Adventurer);
    ~Adventurer();
    Item& getItemAt(const int& index);
    void setItemAt(const int& index, const Item&);
    void initializeItems();
};
#endif

#ifndef ITEM_H
#define ITEM_H
#include <string>
using namespace std;

class Adventurer;

class Item {
private:
    string name;
    unsigned int amount; //number of this type of item
public:
    friend class Adventurer;
};
#endif
```

- 3.1 Which operator can be overloaded to replace the get and set functions in the Adventurer class? Name the operator and also write down the code to overload this operator. Provide necessary error checking. / Watter operator kan oorgelaai word om die get en set funksies hieronder te vervang? Skryf die naam van die operator neer en skryf ook die kode neer om hierdie operator te oorlaai. Verskaf die nodige fouttoetsing. (6)

The [] operator needs to be overloaded.

```
Item& Item::operator[](const int& sub) {
    if (sub < 0 || sub >= items.amount) {
        cout << "Error, subscript out of range" << endl;
        exit(0);
    } else {
        return items[sub];
    }
}
```

4 1/2

- 3.2 Write down the code for the initializeItems function. This function must initialise the items member variable. Each element of the array should be initialised using sensible default values. The array size must be set to 20. / Skryf die kode neer vir die initializeItems funksie. Hierdie funksie moet die items veranderlike inisialiseer. Elke element van die skikking moet geïnisialiseer word met waardes wat sin maak. Aanvaar dat die skikking 'n grootte van 20 het. (4)

```
void Adventurer::initializeItems() {
    items = new Item[20];
    for (int i = 0; i < 20; i++) {
        items[i] = nullptr;
    }
}
```

2

- 3.3 Write down the code that is required in the `Adventurer.h` file to overload the `ostream` operator. / Skryf die kode neer wat benodig word in die `Adventurer.h` lêer om die `ostream` operator te oorlaai. (5)

`ostream operator << (ostream &, const Adventurer &);`

2

- 3.4 Write down the code to overload the `ostream` operator for the `Adventurer` class, i.e. implement the `ostream` operator. All member variables of the `Adventurer` class should be printed out to the screen. / Skryf die kode neer om die `ostream` operator vir die `Adventurer` klas te oorlaai, i.e. die funksie se kode. Alle veranderlikes van die `Adventurer` klas moet op die skerm vertoon word. (5)

`ostream operator << (ostream &strm, const Adventurer &obj)`

`{`
`strm << obj.name << "has" << obj.health << "health";`
~~`strm << "List of items" << endl;`~~
~~`for (int i = 0; i < obj.items.size(); i++)`~~
~~`{`~~
~~`strm << obj.items[i] << " ";`~~
~~`}`~~
`strm << obj.items;`
`return strm;`
`}`

- 3.5 Write down the code to overload the post-decrement operator for the `Adventurer` class. This operator should decrement the value of `health`. / Skryf die kode neer om die post-decrement operator te oorlaai. Hierdie operator moet die waarde van `health` verminder. (5)

`Adventurer & Adventurer::operator --(int) {`
`int *temp = new Adventurer(*this);`
~~`*health--;`~~
`if (health > 0)`
`health--;`
`return *temp;`
`}`

5 - 1/2

Question 4

4.1 For all parts of this question, refer to the code below. / Vir alle dele van hierdie vraag, verwys na die kode hieronder. (13 marks)

```
#ifndef TROLL
#define TROLL
#include "Skill.h"
class Troll {
private:
    std::string name;
    Skill* skills;
    unsigned int numberOfSkills; //maximum number of skills
public:
    Troll(int numSkills); // initialise the skills array
    Troll(const Troll&);
    ~Troll();
    std::string getName() const;
    void setName(std::string n);
    unsigned int getNumberOfSkills() const;
    bool setSkillAt(const unsigned int& ind, const Skill& s);
    Skill& getSkillAt(const unsigned int&);
};
#endif

#ifndef GOBLIN
#define GOBLIN
#include "Skill.h"
class Goblin {
private:
    std::string name;
    Skill* skills;
    unsigned int numberOfSkills; //maximum number of skills
public:
    Goblin(std::string, int);
    Goblin(const Goblin& g);
    ~Goblin();
    std::string getName() const;
    Skill& operator[](const unsigned int&);
};
#endif

#ifndef SKILL
#define SKILL
#include <string>
class Skill {
private:
    std::string name;
    unsigned int level;
public:
    Skill(std::string, int);
    ~Skill();
    string getName() const;
    void setName(std::string n);
    unsigned int getLevel() const;
    void setLevel(unsigned int l);
};
#endif
```

- a) Write down the statements that are missing from the Goblin.h and Troll.h files necessary to cast a Goblin object to a Troll object and vice versa. / Skryf die kode neer wat nog kort in die Goblin.h en Troll.h lêers om 'n Goblin objek na 'n Troll objek te om te skakel, en omgekeerd. (6)

in Troll.h

~~Troll::operator Goblin();~~
~~Troll::operator Goblin(Goblin g);~~

(4)

in Goblin.h

Goblin::operator Troll();
~~Goblin::operator Troll(Troll t);~~

- b) Implement the function to enable an implicit cast from a Goblin object to a Troll object. / Skryf die funksie en sy kode neer om 'n Goblin objek implisiet na 'n Troll objek te om te skakel. (7)

Goblin::operator Troll() {
 Troll troll(numberOfSkills);
 troll.setName(name);
 for (int i=0; i<numberOfSkills; i++) {
 setSkillAt(i, skills[i]);
 }
 return troll;
}

(7)

Question 5 (26 marks)

- 5.1 Define and state the differences between overloaded, redefined and overridden functions. / Definieer en noem die verskille tussen oorlaaiing, herdefinieering en oorlaaide funksies. (5)

Overloaded Functions - Functions with the same name but with different parameter list types; where order is important. 3/12

Redefined Functions are statically bound functions defined within a child class; that being a class that has inherited traits from another class. Within the scope of the child class the new defined function has in a sense replaced the implementation of that of the parent class. 1/1/2

Overridden Functions are dynamically bound functions; that being function bound during runtime (virtual functions) are being overwritten. 1/1/2

- 5.2 Assuming all relevant headers and namespaces have been included, what is the output of the following code segment? / Aanvaar alle relevante "headers" en naamspasies is ingesluit, wat is die afvoer van die volgende kodesegment? (3)

```
class Animal {
public:
    Animal() { cout << "A+" << endl; }
    ~Animal() { cout << "A-" << endl; }
};

class Zebra : public Animal {
public:
    Zebra() { cout << "Z+" << endl; }
    ~Zebra() { cout << "Z-" << endl; }
};

int main()
{
    Zebra z;
    Animal a;
    return 0;
}
```

A+ ✓
Z+ ✓
A+ ✓
A- ✓
Z- ✓
A- ✓

3

5.3 The following classes form an inheritance hierarchy: / Die volgende klasse vorm 'n oorerwingshiërargie:

```
class Top {
public: void foo();
};

class Middle: private Top {
protected: void bar();
};

class Bottom: protected Middle {
public: void qux();
};
```

Say whether each of the following code snippets are valid. The effect of each snippet does not carry over to the next. Assume that, if a function is called in a snippet, that function has a definition elsewhere. / Sê of elkeen van die volgende kode-segmente geldig is of nie. Die effek van elke kode-segment dra nie oor na die volgende segment nie. Aanvaar dat, indien 'n funksie in een kode-segment geroep word, dit wel op 'n ander plek gedefinieer is.

a) `void x() {
 Top t;
 t.qux();
}`

(1)

Invalid

✓

b) `void Middle::bar() {
 foo();
}
void x() {
 Middle m;
 m.bar();
}`

(1)

2

Invalid

✓

c) `void Bottom::qux() {
 bar();
}`

(1)

Invalid

5.4 For all parts of this question, refer to the code below. / Vir alle dele van hierdie vraag, verwys na die kode hieronder.

```
class Employee {
    private:
        string name;
        float salary;
    public:
        Employee(string n, float s);
        ~Employee();
        void work();
};
```

```
class HR : public Employee {
    public:
        HR(string n, float s);
        ~HR();
        void discipline(HR& person);
};
```

```
class Finance : public Employee {
    public:
        Finance(string n, float s);
        ~Finance();
        void viewAccounts();
};
```

```
class Bookkeeper : public Finance {
    public:
        Bookkeeper(string n, float s);
        ~Bookkeeper();
        void recordTransactions();
};
```

```
class Accountant : public Finance {
    private:
        string* statements;
        float budget;
    public:
        Accountant(string n, float s, float b);
        ~Accountant();
        void paySalary();
        void work();
};
```

a) Indicate if the following code snippets are valid or invalid. If invalid, provide a reason for why it is invalid. (8)
/ Dui aan of die volgende kodesegmente geldig is of nie. Indien dit ongeldig is, gee 'n rede hoekom dit ongeldig is.

(i) `HR* emp = new Employee("Mary", 6000);`

Invalid, a parent class cannot be assigned to a child class. ✓ 2

(ii) `void Accountant::paySalary()`
`{`
`cout << "Paying " << name << " : R" << salary << endl;`
`}`

Invalid, 'name' and 'salary' are private member variables of the Employee class. Accountant publicly inherits from Finance which in turn publicly inherits from Employee therefore none of the private member variables are accessible to Accountant.

(iii) `Employee** employees = new Employee*[2];`
`employees[0] = new Bookkeeper("Thando", 6000);`
`employees[1] = new HR("Mary", 6000);`

Valid

(iv) `Employee* emp = new Bookkeeper("John", 6000);`
`emp->recordTransactions();`

Invalid, the new Bookkeeper was sliced the moment it was assigned to the emp pointer. i.e. it cannot use the recordTransactions() function as it has been polymorphed.

b) Implement the Accountant constructor such that it initializes all required members. The statements list should be initialized to null. / Implementeer die Accountant konstrueerder sodat dit al die vereiste lede initialiseer. Die statements lys moet ook geïnitialeer word na null toe. (3)

`statements = NULL;`
`budget = 0;`

c) Rewrite HR's discipline function declaration such that an HR person can discipline any HR or Finance person. / Herskryf HR se discipline funksieverklaring sodat 'n HR persoon enige HR of Finance persoon kan disiplineer. (1)

`void discipline(Employee & person);`

- d) Consider the code snippet below. / Beskou die kodesegment hieronder. (1)

```
int main()
{
    Employee* acc = new Accountant("Jane", 6500, 5000000);
    //do interesting stuff...
    delete acc;
    return 0;
}
```

What do you need to do within the `Employee` class definition to ensure that there are no memory leaks when the `acc` object gets deleted? Write the necessary statement below. / Wat moet jy doen met die `Employee` klasdefinisie om te verseker dat daar geen geheuelekasies is wanneer die `acc` objek uitgevee word nie? Skryf die nodige stelling hieonder neer.

`virtual ~Employee();` ✓ 1

- e) Implement `void Accountant::work()` such that it invokes the `paySalary` function followed by its ancestor's `work` function. / Implementeer `void Accountant::work()` sodat dit die `paySalary` funksie roep, gevolg deur sy voorvaders se `work` funksie. (2)

```
void Accountant::work() {
    paySalary();
    Employee::work();
}
```

✓ 2